Lab05

Max Stróżyk

Winter 2024/2025

English translation of the Polish original. Historical results and formulas retained.

Contents

1 Task 1: QR decomposition **2**
- 1.1 Method overview and problem . . . 2
- 1.2 Tests . . . 2
- 1.3 Results and analysis . . . 2
- 1.4 Conclusions . . . 3

2 Task 2: approximation in inner-product spaces **4**
- 2.1 Method overview and problem . . . 4
- 2.2 Solution method . . . 4
- 2.3 Tests . . . 4
- 2.4 Conclusions and analysis . . . 5

3 Task 3: numerical quadrature **6**
- 3.1 Method overview and problem . . . 6
- 3.2 Solution method . . . 6
- 3.3 Tests . . . 6
- 3.4 Conclusions and analysis . . . 7

Project on GitHub

1 Task 1: QR decomposition

1.1 Method overview and problem

QR decomposition factors a matrix A into a product of two matrices:

$$A = QR,$$

where:

- Q is orthogonal ($Q^T Q = I$),
- R is upper triangular.

The decomposition uses the Gram–Schmidt process to orthogonalize the columns of A. It can be written as:

$$q_1 = \frac{a_1}{||a_1||}, \quad q_k = \frac{a_k - \sum_{i=1}^{k-1} \langle a_k, q_i \rangle q_i}{||a_k - \sum_{i=1}^{k-1} \langle a_k, q_i \rangle q_i||}.$$

To assess accuracy, we measure the orthogonality error of Q, defined as the largest inner product between distinct vectors q_i and q_j, where $i \neq j$:

$$E = \max\{\langle q_i, q_j \rangle \ : \ i \neq j\}.$$

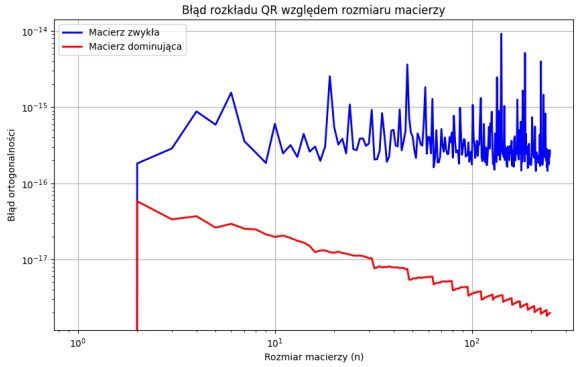
Repeated QR decomposition can reduce orthogonality errors.

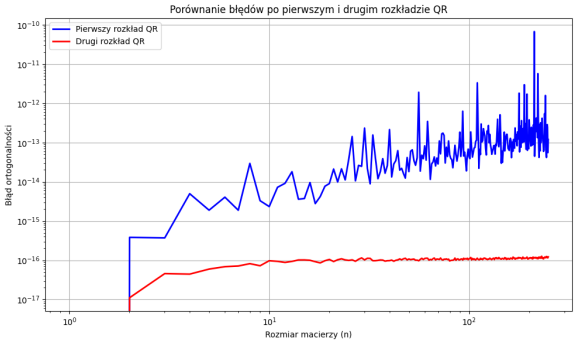
1.2 Tests

The tests examined orthogonality error versus matrix size n for two matrix types, taking the maximum inner product between different columns:

- random matrices,
- diagonally dominant matrices.

The effect of a second QR decomposition on reducing orthogonality error was also analyzed. Matrix sizes ranged from $n = 1$ to $n = 250$. For each n, 10 random matrices were generated and results were averaged.





1.3 Results and analysis

- Orthogonality error depends differently on matrix size n for random and diagonally dominant matrices.
- A second QR decomposition substantially reduces orthogonality errors, as shown in the plot.

1.4 Conclusions

- Repeated QR decomposition can substantially reduce orthogonality errors, which matters in applications requiring high precision.
- Errors of order 10^{-14} are negligible for most applications, but demanding computations may require additional accuracy-improvement steps.

2 Task 2: approximation in inner-product spaces

2.1 Method overview and problem

The task is to find the optimal approximation h^* of a function f in an inner-product space V by minimizing distance in a specified norm:

$$\|h^* - f\| = \inf\{\|u - f\| : u \in U\},$$

assuming U is a subspace of V. Approximation-theory results imply:

- The solution h^* exists and is unique.
- h^* is optimal if and only if $f - h^*$ is orthogonal to U, that is, $\langle f - h^*, u\rangle = 0$ for every $u \in U$.
- h^* can be expressed as a linear combination of basis elements of U.

2.2 Solution method

Assume the basis of U consists of functions $\{f_i\}_{i\in I}$. Orthogonality of $f - h^*$ to U gives:

$$\langle f - h^*, f_i\rangle = 0 \quad \text{for every } i \in I.$$

This gives a unique representation of h^*:

$$h^* = \sum_{i\in I} a_i f_i.$$

The coefficients a_i of the approximation h^* in the basis of U can therefore be found by solving:

$$\langle f, f_i\rangle = \sum_{j\in I} a_j \langle f_j, f_i\rangle.$$

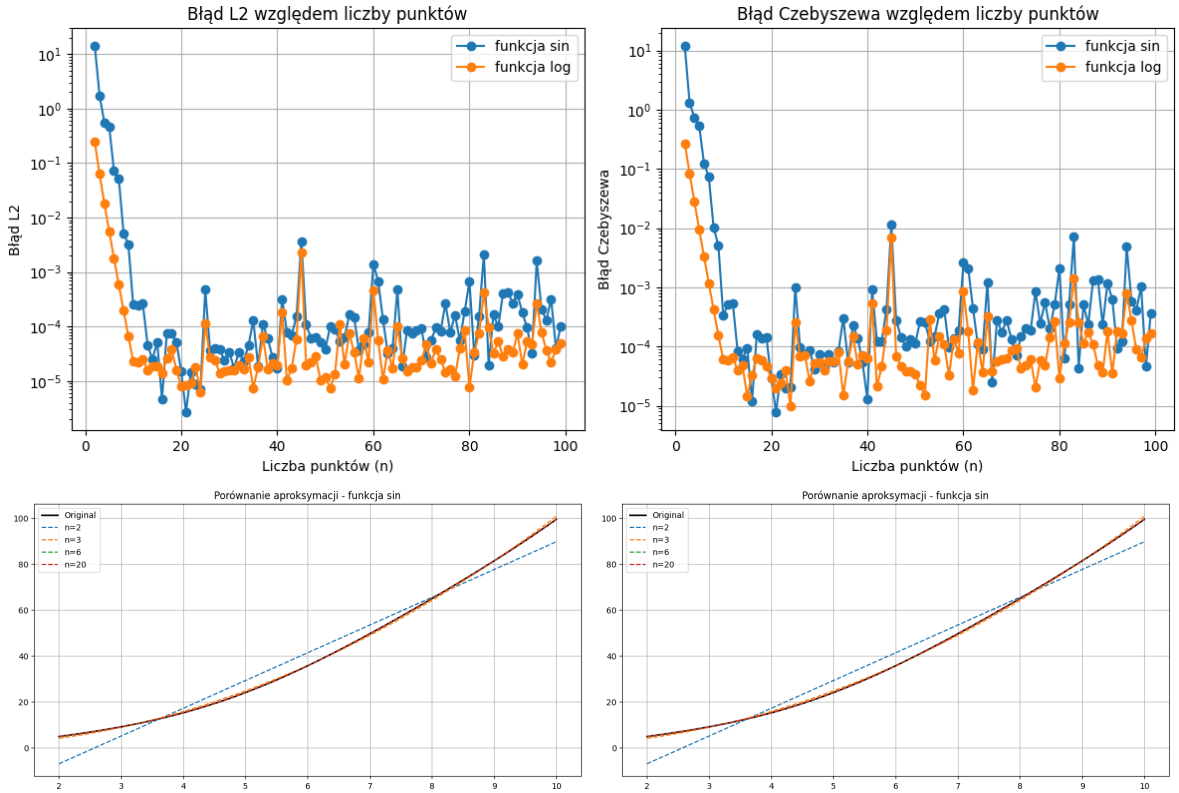
in matrix form $Aa = b$:

$$\begin{bmatrix} \langle f_1, f_1\rangle & \langle f_2, f_1\rangle & \cdots & \langle f_n, f_1\rangle \\ \langle f_1, f_2\rangle & \langle f_2, f_2\rangle & \cdots & \langle f_n, f_2\rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle f_1, f_n\rangle & \langle f_2, f_n\rangle & \cdots & \langle f_n, f_n\rangle \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \langle f, f_1\rangle \\ \langle f, f_2\rangle \\ \vdots \\ \langle f, f_n\rangle \end{bmatrix}$$

2.3 Tests

Approximation quality was evaluated as follows:

- Approximation errors were compared in the Chebyshev and L^2 metrics.
- Convergence to the original function was checked for different polynomial degrees.



2.4 Conclusions and analysis

- Distance minimization with h^* gives smaller errors in both metrics than interpolation: previously 10^{-1}, now $10^{-4} - 10^{-6}$.
- Functions approximated by h^* approach the original function faster, as the convergence plots show: previously around $25 - 30$, here $10 - 20$.

Computational-complexity comparison

- **Lagrange interpolation:** $O(n^2)$.
- **Newton's method:** $O(n^2)$.
- **Distance minimization h^*:** $O(n^3)$, because a linear system must be solved by the LU method.

Conclusions

- Minimization with h^* gives better accuracy and faster convergence than the previous methods.
- Approximation accuracy increases exponentially with polynomial degree, allowing effective approximation even of functions with complex structure.

3 Task 3: numerical quadrature

3.1 Method overview and problem

Numerical quadrature approximates definite integrals by a weighted sum. The task is to find weights Z_i and points L_i satisfying:

$$\int_a^b f(x)dx \approx \sum_{i=1}^{n} f(L_i)Z_i,$$

The points L_i are evenly spaced over $[a, b]$. The method is derived by requiring the resulting linear system to be exact for polynomials of degree $n-1$. For greater precision, $[a, b]$ can be divided into smaller subintervals. These methods form the basis of algorithms for solving problems [sentence unfinished in the original].

3.2 Solution method

1. We want to solve a problem of the form:

$$I(f) = \sum_{i=0}^{n-1} Z_i \cdot f(L(i))$$

2. Form the system: construct the appropriate matrix and right-hand-side vector:

$$A_{ij} = L_j^i, \quad B_i = \frac{b^i - a^i}{i}.$$

3. Solve the linear system $AZ = B$, described by:

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ L_1 & L_2 & \cdots & L_n \\ L_1^2 & L_2^2 & \cdots & L_n^2 \\ \vdots & \vdots & \ddots & \vdots \\ L_1^{n-1} & L_2^{n-1} & \cdots & L_n^{n-1} \end{bmatrix} \begin{bmatrix} Z_1 \\ Z_2 \\ \vdots \\ Z_n \end{bmatrix} = \begin{bmatrix} b-a \\ \frac{b^2-a^2}{2} \\ \frac{b^3-a^3}{3} \\ \vdots \\ \frac{b^n-a^n}{n} \end{bmatrix}.$$

4. Generate interpolation points: define evenly spaced points as:

$$L_i = a + i\frac{b-a}{n-1}, \quad i = 0, 1, \ldots, n-1.$$

3.3 Tests

Comparative integration tests covered:

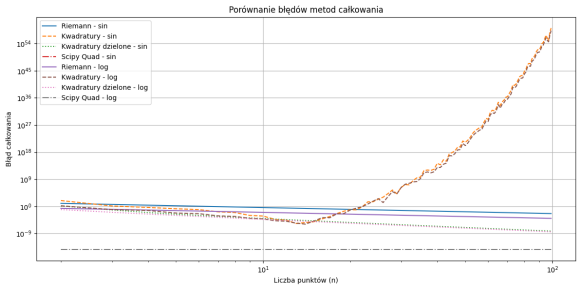
- Riemann rectangle sums.
- Classical quadrature with evenly spaced interpolation points.
- Composite quadrature, dividing the interval into smaller parts.
- The `quad` function from `scipy`.

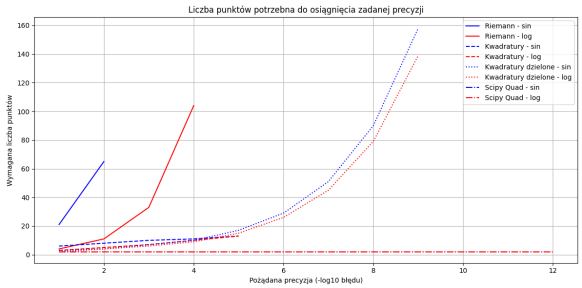
Tests used the interval $[2, 10]$ and functions $\sin(x) + x^2, \ln(x)$:

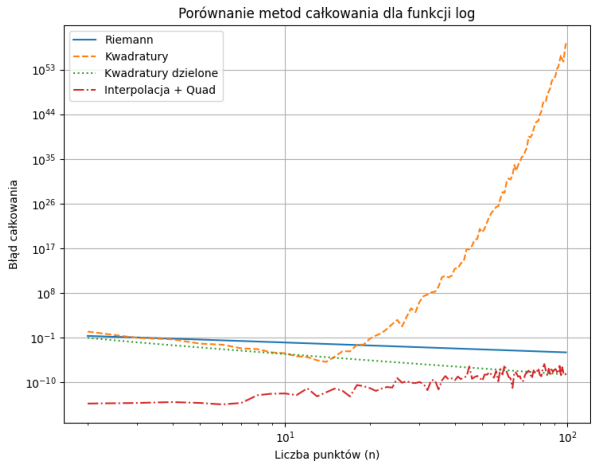
1. Integration error versus interpolation-point count, from 2 to 100.

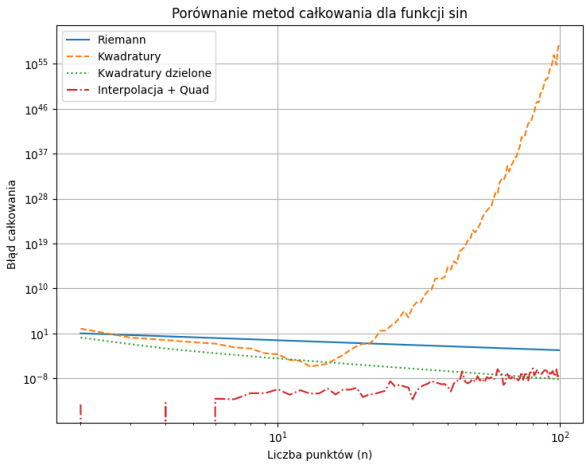
2. Points needed to attain a specified precision, with a maximum interpolation count of $25 * (b - a)$.

3. Convergence of the sum for the interpolated and integrated case, where the integral is known, compared with its approximation.









3.4 Conclusions and analysis

- Riemann sums are least efficient and cannot attain the desired precision even with very many points.
- Ordinary quadrature is less effective than composite quadrature. It also performs much worse computationally because solving the matrix system is the most expensive operation.
- Composite quadrature considerably improves accuracy but is slower and less accurate than the built-in `quad`.
- `quad` performs best because it selects nonuniform interpolation points and uses Legendre polynomials, enabling accurate approximation of difficult functions.
- Computing the interpolating polynomial beforehand is beneficial. The error then depends only on the approximation.

Computational complexity :

- Riemann rectangle sums: $O(n)$.
- Classical quadrature: $O(n^3)$, from solving the linear system.
  - Matrix construction: $O(n^2)$.
  - Linear-system solution: $O(n^3)$, using LU.
  - Therefore the complexity is $O(n^3)$.

- Composite quadrature: $O(k \cdot n^3)$, where k is the subinterval count and n is the number of points per subinterval; here $n = 3$.
- `quad`: an adaptive method with nonuniform points. Its complexity depends on the function, but it is efficient in practice because of its use of Legendre polynomials.

References

[a20, 2022] (2022). Lapack examples.

[Embree, 2016] Embree, M. (2016). Math/cs 5466 - virginia tech - spring 2016. lecture 24.

[Jabłoński,] Jabłoński, Z. Lectures from numerical method.

[Miller et al.,] Miller, J., Wąs, J., and Wprowadzenie, . 3. kwadratury.

[Patterson, 1968] Patterson, T. N. L. (1968). The optimum addition of points to quadrature formulae. *Mathematics of Computation*, 22:847.